
boolean



Immutabilità dello stato

lavorare con oggetti ed array

Immutabilità dello stato

Come funziona il rendering di React ?

Quando viene invocata la funzione **setState**, React determina se è necessario aggiornare la UI del componente **confrontando** lo **stato corrente** con il **nuovo stato** passato come argomento.

Questo è il motivo per cui **NON** è possibile modificare direttamente lo stato.

Nel caso di oggetti ed array, React non è in grado di controllarne il contenuto interno. Quindi se cambiassimo un elemento di un array o la proprietà di un oggetto, React non aggiornerebbe la UI.

Proviamo a veriricarlo facendo una semplice todo list

Immutabilità dello stato

Immaginiamo di avere un'array di stringhe che rappresenta una lista di cose da fare.

Nel nostro form aggiungiamo un nuovo elemento... e poi inviamo il form.

Nella funzione `addTask`, inseriamo il nuovo task con il metodo `push`.

Se in basso stiamo stampando la nostra lista di tasks... noteremo che anche se abbiamo aggiunto un nuovo elemento, **la lista non si aggiorna!**

```jsx
import { useState } from 'react'

function App() {

  const [tasks, setTasks] = useState(['Fare la spesa']);
  const [newTask, setNewTask] = useState('');

  const addTask = e => {
    e.preventDefault();
    tasks.push(newTask);
  }

  return (
    <>
      <form onSubmit={addTask}>
        //....
      </form>

      //lista dei task
      <ul>
        {tasks.map((task, i) => (
          <li key={i}>{task}</li>
        ))}
      </ul>
    </>
)
```

Immutabilità dello stato

Come funziona il rendering di React ?

Come abbiamo visto, cambiare gli elementi di un array o di un oggetto non funziona.

Pertanto, per far capire a React che qualcosa è cambiato, dobbiamo creare un array (o un oggetto) nuovo di zecca, **clonando il precedente**!

Successivamente, modifichiamo questo clone con quel che ci interessa e lo passiamo alla funzione **setState** per aggiornare lo stato in modo appropriato.

A quel punto, avendo un oggetto nuovo, React capirà che qualcosa è cambiato ed aggiornerà la UI!

Immutabilità dello stato

Come aggiornare correttamente

Ecco dunque che per riuscire ad aggiornare la UI potremmo:

- clonare l'array precedente tramite **spread**
- usare `setTasks()` per aggiornare lo state

Ed ecco che ora la lista si aggiornerà anche in pagina!

Possiamo anche fare tutto su un rigo!

```
1  import { useState } from 'react'
2
3  function App() {
4
5    const [tasks, setTasks] = useState(['Fare la spesa']);
6    const [newTask, setNewTask] = useState('');
7
8    const addTask = e => {
9      e.preventDefault();
10     const updatedTasks = [...tasks, newTask];
11     setTasks(updatedTasks);
12   }
13
```

```
1  import { useState } from 'react'
2
3  function App() {
4
5    const [tasks, setTasks] = useState(['Fare la spesa']);
6    const [newTask, setNewTask] = useState('');
7
8    const addTask = e => {
9      e.preventDefault();
10     setTasks([...tasks, newTask]);
11   }
12
```

Immutabilità dello stato

Rimuovere un elemento da un Array

E se volessimo rimuovere un elemento dal menù?

Innanzitutto inseriamo un bottone di fianco ai nostri task ed una funzione che reagisca al loro click.

Anche la rimozione di un elemento da un array non viene percepita come una modifica da React.

Pertanto, creiamo un nuovo array nel quale non è presente l'elemento che volevamo eliminare, utilizzando il metodo **filter!**

```jsx
<ul>
  {tasks.map((task, i) => (
    <li key={i}>
        <span>{task}</span>
        <button onClick={() => removeTask(i)}>
          Elimina
        </button>
    </li>
  ))}
</ul>
```

```jsx
// ...
const [tasks, setTasks] = useState(['Fare la spesa']);

const  removeTask = i => {
  const updatedTasks = tasks.filter((task, index) => {
   return index !== i
  }));
  setTasks(updatedTasks);
}
```



LIVE CODING



ESERCIZIO